

Examen de Sistemas Inteligentes

Nombre y apellidos: _____

Fecha: _____

Pregunta 1 (test)

PREGUNTA DE EXAMEN: Algoritmo A*

Enunciado

Considere la ejecución del algoritmo A* sobre un mapa donde se exploran nodos usando una cola de prioridad (heap). Durante la expansión de un nodo actual, se genera un hijo con posición válida que ya existe en la listaFrontera con un valor g anterior de 8.5. El nuevo camino hacia este hijo calcula una nueva_g = 7.2.

Según la lógica del código, ¿qué ocurre en esta situación?

Opciones

A) El nodo hijo se añade nuevamente a la listaFrontera porque nueva_g < g_antigua, creando duplicados, y después ambas copias se procesarán según su f(n).

B) Se descarta el nuevo camino y se mantiene el nodo existente en la listaFrontera sin cambios, ya que el algoritmo solo expande cada nodo una única vez.

C) Se modifica el nodo existente en la listaFrontera actualizando su g, su h, su padre y su valor calórico, porque se encontró un camino mejor hacia esa posición.

D) Se reemplaza completamente el nodo antiguo por uno nuevo, eliminando toda la información anterior incluida su coste calórico acumulado.

Código (a_estrella):

```
def a_estrella(mapa, origen, destino, camino):
    heuristica = octile
    listaInterior = [] # vacía inicialmente
    listaFrontera = []
    nodo_origen = Nodo(origen)
    nodo_origen.calculateH(heuristica, destino)

    heapq.heappush(listaFrontera, nodo_origen) # nodo inicial en la lista frontera
    #print(f"Nodo origen: {nodo_origen.getCasilla().getFila()}, {nodo_origen.getCasilla().getCol()}")

    while listaFrontera != []: # mientras listafrontera no esté vacía
        actual = listaFrontera[0] # obtenemos el nodo de listafrontera con menor f(n) = g(n) + h(n)
        #print(f"Actual: {actual.getCasilla().getFila()}, {actual.getCasilla().getCol()}, f(n): {actual.getCost()}, cal: {actual.cal}")
        #print(f"ListaInterior: {[f'{nodo.getCasilla().getFila()}, {nodo.getCasilla().getCol()}' for nodo in listaInterior]}")
        if actual.casilla == destino:
```

Examen de Sistemas Inteligentes

```
listaInterior.append(actual)
printNodosExplorados(mapa.alto, mapa.ancho, listaInterior)
reconstruirCamino(actual, camino) # reconstruir camino con los nodos padre
return (actual.getCost(), actual.cal)

else:
    actual = heapq.heappop(listaFrontera) # borramos el nodo de listafrontera
    listaInterior.append(actual) # lo añadimos a listainterior
    #print(f"Nodo expandido: {actual.getCasilla().getFila()}, {actual.getCasilla().getCol()}")

    # expando el nodo en todas las direcciones, tengo que ver si estas son válidas (no son azul
    marino), y que no estén en la lista anterior
    direcciones = [ # diferentes movimientos y su coste
        (0, -1, 1), # Arriba
        (-1, 0, 1), # Izquierda
        (1, 0, 1), # Derecha
        (0, 1, 1), # Abajo
        (-1, -1, 1.5), # Diagonal superior izquierda
        (1, -1, 1.5), # Diagonal superior derecha
        (-1, 1, 1.5), # Diagonal inferior izquierda
        (1, 1, 1.5) # Diagonal inferior derecha
    ]

    for dx, dy, cost in direcciones: # para cada hijo m de n
        nueva_pos = actual.getCasilla() + Casilla(dx, dy)
        g_antigua = [0]
        if (not nueva_pos.validarPosicion(mapa)) or (existe_nodo(listaInterior, nueva_pos,
        g_antigua)): # QUE NO ESTÉ EN LISTA INTERIOR
            continue

        nueva_g = actual.getG() + cost
        nueva_cal = actual.cal + valorCalorico(nueva_pos, mapa)
        hijo = Nodo(nueva_pos)
        #print(f" Evaluando hijo: {hijo.getCasilla().getFila()}, {hijo.getCasilla().getCol()}, g:
        {nueva_g}, cal: {nueva_cal}")

        if not existe_nodo(listaFrontera, hijo.getCasilla(), g_antigua): # si m no está en la
        listafrontera
            hijo.setG(nueva_g)
            hijo.calculateH(heuristica, destino)
            hijo.cal = nueva_cal
            hijo.setPadre(actual)
            heapq.heappush(listaFrontera, hijo)
            #print(f" Añadido a listaFrontera: {hijo.getCasilla().getFila()},
            {hijo.getCasilla().getCol()}, f(n): {hijo.getCost()}, cal: {hijo.cal}")

            elif nueva_g < g_antigua[0]: # sino si la nueva g es mejor que la g que tiene el nodo en la
            lista frontera entonces este camino es mejor
                modifica_nodo(listaFrontera, hijo.getCasilla(), nueva_g, actual, nueva_cal) #
                recalculamos f y g y el padre del nodo m y modificamos el nodo ya existente en la lista frontera
                #print(f" Modificado en listaFrontera: {hijo.getCasilla().getFila()},
                {hijo.getCasilla().getCol()}, f(n): {hijo.getCost()}, cal: {hijo.cal}")
```

Examen de Sistemas Inteligentes

```
return (-1, -1)
```

Pregunta 2 (test)

PREGUNTA DE EXAMEN: Función modifica_nodo

Enunciado:

Considera la siguiente secuencia de ejecución sobre un heap de nodos (usado en algoritmos de búsqueda como A*):

```
heap = [nodo1, nodo2, nodo3] # nodo2.getCasilla() == 5
```

```
modifica_nodo(heap, 5, 15, padre_x, 22)
```

Luego se llama nuevamente:

```
modifica_nodo(heap, 7, 10, padre_y, 18)
```

¿Cuál es el impacto más significativo de llamar a `heapq.heapify()` dentro de la función en lugar de usar operaciones puntuales como `heapq.heapreplace()` o `heapq.heappushpop()`?

A) Garantiza que la propiedad de heap se mantiene, pero tiene complejidad $O(n)$ en lugar de $O(\log n)$, lo que es ineficiente si se modifica un nodo frecuentemente.

B) Optimiza la búsqueda lineal inicial del nodo dentro del heap reduciendo su complejidad a $O(\log n)$.

C) Evita que nodos duplicados se inserten en el heap durante la modificación.

D) Mejora el rendimiento porque `heapify()` es más rápida que reconstruir el heap desde cero.

Código (modifica_nodo):

```
def modifica_nodo(heap, casilla, nueva_g, actual, nueva_cal):
    for nodo in heap:
        if nodo.getCasilla() == casilla:
            nodo.setG(nueva_g)
            nodo.setPadre(actual)
            nodo.cal = nueva_cal
            heapq.heapify(heap)
            return True
    return False
```

Pregunta 3 (traza)

PREGUNTA DE EXAMEN: TRAZA DE EJECUCIÓN

Enunciado

Considera la siguiente estructura de datos:

Examen de Sistemas Inteligentes

- Una clase Casilla con métodos getFila() y getCol() que devuelven coordenadas enteras.
- Una clase Nodo que contiene un atributo padre (referencia a otro Nodo o None) y un método getCasilla() que devuelve una Casilla.
- Una matriz camino de 5×5 inicialmente llena de valores '.'.

Se ejecuta el siguiente código:

```
reconstruirCamino(nodo_final, camino)
```

Donde nodo_final es una cadena enlazada de nodos construida así:

- Nodo 4: Casilla(fila=4, col=3), padre=None
- Nodo 3: Casilla(fila=2, col=2), padre=Nodo4
- Nodo 2: Casilla(fila=1, col=3), padre=Nodo3
- Nodo 1: Casilla(fila=0, col=2), padre=Nodo2

Se pasa nodo_final=Nodo1 y una matriz camino vacía (todos elementos son '.').

Pregunta: ¿Cuál será el contenido de la matriz camino después de ejecutar la función? Especifica en qué posiciones se colocan las 'X' y explica el recorrido realizado por el bucle while.

Código (reconstruirCamino):

```
def reconstruirCamino(nodo_final, camino):  
    while nodo_final is not None:  
        camino[nodo_final.getCasilla().getFila()][nodo_final.getCasilla().getCol()] = 'X'  
        nodo_final = nodo_final.padre
```

Pregunta 4 (test)

PREGUNTA DE EXAMEN: Sistemas Inteligentes (3º Grado)

Enunciado:

Considera la siguiente secuencia de ejecución de la función valorCalorico:

```
casilla = Casilla(fila=2, col=3)  
mapa.getCelda(2, 3) devuelve 5  
resultado = valorCalorico(casilla, mapa)
```

Posteriormente, se ejecuta otra llamada:

```
casilla2 = Casilla(fila=1, col=1)  
mapa.getCelda(1, 1) devuelve 7  
resultado2 = valorCalorico(casilla2, mapa)
```

¿Cuál será el estado de resultado y resultado2 tras ejecutar ambas llamadas?

Examen de Sistemas Inteligentes

A) resultado = 6 y resultado2 = -1

B) resultado = 5 y resultado2 = 7

C) resultado = 6 y resultado2 = 4

D) resultado = -1 y resultado2 = -1

Código (valorCalorico):

```
def valorCalorico(casilla, mapa):
    valorCasilla = mapa.getCelda(casilla.fila,casilla.col)
    valorCalorico = -1

    if valorCasilla == 0:
        valorCalorico = 2
    elif valorCasilla == 4:
        valorCalorico = 4
    elif valorCasilla == 5:
        valorCalorico = 6
    else:
        return -1 #es una pared
    return valorCalorico
```

Pregunta 5 (traza)

Pregunta de Examen: Traza de Ejecución de printNodosExplorados

Enunciado

Considera la ejecución de la siguiente llamada a la función printNodosExplorados:

```
class Nodo:
```

```
    def __init__(self, fila, col):
        self.casilla = type('Casilla', (), {'fila': fila, 'col': col})()
```

```
lista = [
    Nodo(0, 1),
    Nodo(1, 0),
    Nodo(1, 2),
    Nodo(2, 1)
```

```
]
```

```
printNodosExplorados(3, 3, lista)
```

Pregunta: ¿Cuál es exactamente la salida impresa por esta llamada? Escribe la salida línea por línea,

Examen de Sistemas Inteligentes

indicando explícitamente los espacios y alineación generados por el formato `f"{num:<3}"`.

Código (printNodosExplorados):

```
def printNodosExplorados(alto, ancho, lista_interior):
    matriz = []
    for i in range(alto):
        matriz.append([])
        for j in range(ancho):
            matriz[i].append(-1)

    for i in range(len(lista_interior)):
        casillaNodo = lista_interior[i].casilla
        matriz[casillaNodo.fila][casillaNodo.col] = i

    for fila in matriz:
        print(' '.join(f"{num:<3}" for num in fila))
```

Pregunta 6 (test)

PREGUNTA DE EXAMEN: Algoritmo A* con Parámetro Epsilon

Enunciado

Considere la ejecución del algoritmo `a_estrella_eps` en un mapa donde se exploran varios nodos. En un momento dado de la ejecución:

- `listaFrontera` contiene 5 nodos con valores de coste $f(n) = g(n) + h(n)$ iguales a: [10, 12, 15, 18, 22]
- `epsilon` = 3
- `min_f` = 10 (el primer elemento del heap)

¿Cuál será el comportamiento correcto al construir `listaFocal` y seleccionar el próximo nodo a expandir?

- A) `listaFocal` contendrá todos los 5 nodos, y se seleccionará aquel con mayor valor de cal (coste calórico).
- B) `listaFocal` contendrá solo los nodos con $f(n) \leq 30$, y entre ellos se seleccionará el que tenga menor valor de cal (coste calórico).
- C) `listaFocal` contendrá solo los nodos con $f(n) \leq 10$, y se seleccionará el primero del heap (criterio FIFO).
- D) `listaFocal` contendrá los 4 primeros nodos ordenados por $f(n)$, descartando automáticamente el nodo con $f(n) = 22$.

Código (`a_estrella_eps`):

```
def a_estrella_eps(mapa, origen, destino, camino, epsilon=3):
    heuristica = octile
    listaInterior = [] # vacía inicialmente
    listaFrontera = []
    listaFocal = []
    nodo_origen = Nodo(origen)
```

Examen de Sistemas Inteligentes

```
nodo_origen.calculateH(heuristica, destino)

heapq.heappush(listaFrontera, nodo_origen) # nodo inicial en la lista frontera

while listaFrontera:
    min_f = listaFrontera[0].getCost() #es un heapmin entonces el primer elemento es el menor
    listaFocal = [nodo for nodo in listaFrontera if nodo.getCost() <= (epsilon) * min_f] #sublista de
frontera: listafocal
    actual = min(listaFocal, key=lambda nodo: nodo.cal) #seleccionar nodo con menor coste calórico de
lista focal(nueva heurística)
    listaFrontera.remove(actual)
    heapq.heapify(listaFrontera)

    if actual.casilla == destino:
        listaInterior.append(actual)
        printNodosExplorados(mapa.alto, mapa.ancho, listaInterior)
        reconstruirCamino(actual, camino) # reconstruir camino con los nodos padre
        return (actual.getCost(), actual.cal)
    else:
        listaInterior.append(actual) # lo añadimos a listainterior

# Expando el nodo en todas las direcciones
direcciones = [
    (0, -1, 1), (1, 0, 1), (0, 1, 1), (-1, 0, 1),
    (-1, -1, 1.5), (1, -1, 1.5), (-1, 1, 1.5), (1, 1, 1.5)
]

for dx, dy, cost in direcciones:
    nueva_pos = actual.getCasilla() + Casilla(dx, dy)
    g_antigua = [0]
    if not nueva_pos.validarPosicion(mapa) or existe_nodo(listaInterior, nueva_pos, g_antigua):
        continue

    nueva_g = actual.getG() + cost
    nueva_cal = actual.cal + valorCalorico(nueva_pos, mapa)
    hijo = Nodo(nueva_pos)

    if not existe_nodo(listaFrontera, hijo.getCasilla(), g_antigua):
        hijo.setG(nueva_g)
        hijo.calculateH(heuristica, destino)
        hijo.cal = nueva_cal
        hijo.setPadre(actual)
        heapq.heappush(listaFrontera, hijo)
    elif nueva_g < g_antigua[0]:
        modifica_nodo(listaFrontera, hijo.getCasilla(), nueva_g, actual, nueva_cal)

return (-1, -1)
```